

实验报告：BIRENSUPA编程入门——向量加法实现

1. 实验目的

本实验旨在初步了解 BIRENSUPA 编程模型，掌握基本的 GPU 异构编程流程。具体目标包括：

- 理解主机 (Host) 与设备 (Device) 的内存管理机制。
- 掌握核心运行时 API (如 `suMallocDevice`, `suMemcpy`, `suLaunchKernel`) 的使用。
- 编写并运行一个简单的并行向量加法核函数 (Kernel)。

2. 实验原理

2.1 编程模型

BIRENSUPA 采用主机-设备 (Host-Device) 模型。

- **Host (CPU)**: 负责逻辑控制、内存分配和数据传输。
- **Device (GPU)**: 负责高并发的计算任务。该模型将底层硬件细节抽象为线程 (Thread)、计算组等概念，利用 GPU 的大量执行单元 (EU) 进行单指令多线程 (SIMT) 并行计算。

2.2 线程索引计算

在 GPU 并行计算中，每个线程需要计算其操作的数据在全局内存中的唯一索引。

根据 BIRENSUPA 的内置变量，全局索引 idx 的计算公式为：

$$idx = thread_idx.x + block_idx.x \times block_dim.x$$

其中：

- `thread_idx.x`: 线程在当前块内的偏移。
- `block_idx.x`: 当前块在网格中的偏移。
- `block_dim.x`: 一个块中包含的线程总数。

3. 实验步骤与代码分析

实验主要分为两个部分：设备端核函数的编写与主机端控制代码的编写。

3.1 核函数实现 (`vectorAdd`)

核函数是运行在 GPU 上的代码。主要逻辑如下：

1. **计算索引**: 利用上述公式计算当前线程对应的数组下标 `idx`。

2. **边界检查**：由于线程块的大小通常是固定的（如 1024），总线程数往往大于数组实际长度，因此必须判断 `if (idx < length)` 以防止内存越界。
3. **并行计算**：执行 `dst[idx] = src1[idx] + src2[idx]`，每个线程独立计算一个元素的结果。

3.2 主机函数实现 (`vectorAddHostFunc`)

主机函数负责协调整个计算过程，具体步骤如下：

1. **分配显存**：使用 `suMallocDevice` 为源向量 `src1`、`src2` 和目标向量 `dst` 在 GPU 上分配内存。
2. **数据传输 (H2D)**：使用 `suMemcpy` 将初始化好的数据从 CPU 内存拷贝到 GPU 显存。
3. **配置与启动**：
 - 设置 Block 大小为 1024。
 - 计算 Grid 大小： $(length + 1023) / 1024$ ，向上取整以覆盖所有数据。
 - 使用 `suLaunchKernel` 启动核函数。
4. **数据回传 (D2H)**：计算完成后，使用 `suMemcpy` 将结果从 GPU 显存拷贝回 CPU 内存供验证。
5. **资源释放**：使用 `suFree` 释放申请的 GPU 显存，避免内存泄漏。

4. 实验结果

程序运行后，主机端代码对计算结果进行了验证。

- 输入数据：`src1` 全为 2，`src2` 全为 3。
- 预期结果：`dst` 中所有元素应为 5。
- 验证逻辑：遍历数组，若发现任一元素不等于 5，则测试失败。

最终输出：

Plaintext

```
Test pass!
```

5. 总结

本次实验实现了基于 BIRENSUPA 平台的向量加法。实验验证了 GPU 在处理数据并行任务时的优势，即通过开启数百万个线程同时工作，替代了 CPU 的串行循环结构。同时，熟练掌握了 `suMallocDevice`、`suMemcpy` 和 `suLaunchKernel` 等关键 API 的调用顺序与参数配置，为后续开发更复杂的异构计算应用打下了基础。